Exam: University Library System

Create a server-client application for a university library book reservation and borrowing system!

Subtask 1 (4 points)

Create a TCP server-client application. The server represents the **Library System** and stores available books. Each book has: - Book ID (8 characters) - Title (string) - Number of available copies (integer) - Reservation code (6 random alphanumeric characters, generated at server startup)

The client represents a **Student** who wants to borrow books. The student communicates with the library using messages with ':' separator in the following format: `<action>:<book_id>:<reservation_code>`

Where `<action>` can be: - `"check"` - Check if a book is available (only book_id needed, reservation_code should be empty string) - `"borrow"` - Borrow a book with the reservation code

Server responses: - For `check`: `"available:<number_of_copies>"` or `"unavailable"` - For `borrow`: `"success"` if the reservation code is correct and copies are available, otherwise `"failed"`

When a successful borrow happens, decrease the available copies by 1.

The student should request the book_id and action from **standard input**. If borrowing, also request the reservation code. The student can perform multiple actions in one session.

Subtask 2 (2 points)

Choose ONE of the following:

a) The server reads the initial book inventory from a **JSON file**, with the path given as a command-line argument. The JSON format should be:

```
[
  {"book_id": "BOOK0001", "title": "Network Programming", "copies": 3},
  {"book_id": "BOOK0002", "title": "Data Structures", "copies": 5}
]
```

OR

b) The server receives the number of books to generate as a command-line argument and randomly generates book entries with: - Random book IDs (8 uppercase alphanumeric characters) - Titles from a predefined list: `["Algorithms", "Databases", "Operating Systems", "Computer Networks", "Software Engineering"]` - Random number of copies (between 1 and 10)

Subtask 3 (1 point)

The library server should be able to serve multiple students simultaneously using **select**.

Subtask 4 (3 points)

Create a third component: the **Library Notification System** (UDP server).

Modify the library server so that every time a student successfully borrows a book, it sends a notification to the Notification System via UDP with the format: `<timestamp>:<student_ip>:<book_id>:<remaining_copies>`

The Notification System logs all borrowing activities to a **JSON file** (append mode), with the path given as a command-line argument. Each log entry should contain:

```
{
  "timestamp": "2025-01-15T10:30:00",
  "student_ip": "127.0.0.1",
  "book_id": "BOOK0001",
  "remaining_copies": 2
}
```

After logging, the Notification System sends back an `"ACK"` response to the library server.

Bonus Subtask (optional, +2 points)

Implement a **struct-based** message format instead of the ':' separator format for communication between student and library server. The struct should have at least 3 fields (action, book_id, reservation_code).

Grading:

- **Total: 10 points** (12 with bonus)
- **Passing: 5 points (50%)**
- Subtask 1 alone gives 4 points (40%)
- Students need just 1 more point from Subtasks 2-4 to pass

Usage Examples:

```
# Start notification system
python3 notification_server.py 127.0.0.1 6000 borrowing_log.json

# Start library server (option a - from JSON)
python3 library_server.py 127.0.0.1 5000 books.json 127.0.0.1 6000

# Start library server (option b - generate random)
python3 library_server_random.py 127.0.0.1 5000 5 127.0.0.1 6000

# Start student client
python3 student_client.py 127.0.0.1 5000
```